

Computable function

Computable functions are the basic objects of study in computability theory. Informally, a function is *computable* if there is an algorithm that computes the value of the function for every value of its argument. Because of the lack of a precise definition of the concept of algorithm, every formal definition of computability must refer to a specific model of computation.

Many such models of computation have been proposed, the major ones being Turing machines, register machines, lambda calculus and general recursive functions. Although these four are of a very different nature, they provide exactly the same class of computable functions, and, for every model of computation that has ever been proposed, the computable functions for such a model are computable for the above four models of computation.

The Church–Turing thesis is the unprovable assertion that every notion of computability that can be imagined can compute only functions that are computable in the above sense.

Before the precise definition of computable functions, mathematicians often used the informal term *effectively calculable*. This term has since come to be identified with the computable functions. The effective computability of these functions does not imply that they can be *efficiently* computed (i.e. computed within a reasonable amount of time). In fact, for some effectively calculable functions it can be shown that any algorithm that computes them will be very inefficient in the sense that the running time of the algorithm increases exponentially (or even superexponentially) with the length of the input. The fields of feasible computability and computational complexity study functions that can be computed efficiently.

The Blum axioms can be used to define an abstract computational complexity theory on the set of computable functions. In computational complexity theory, the problem of computing the value of a function is known as a function problem, by contrast to decision problems whose results are either "yes" or "no".

Definition

Computability of a function is an informal notion. One way to describe it is to say that a function is computable if its value can be obtained by an effective procedure. With more rigor, a function $f : \mathbb{N}^k \rightarrow \mathbb{N}$ is computable if and only if there is an effective procedure that, given any k -tuple $\mathbf{x}$ of natural numbers, will produce the value $f(\mathbf{x})$.^[1] In agreement with this definition, the remainder of this article presumes that computable functions take finitely many natural numbers as arguments and produce a value which is a single natural number.

As counterparts to this informal description, there exist multiple formal, mathematical definitions. The class of computable functions can be defined in many equivalent models of computation, including

- Turing machines
- General recursive functions
- Lambda calculus
- Post machines (Post–Turing machines and tag machines).
- Register machines

Although these models use different representations for the functions, their inputs, and their outputs, translations exist between any two models, and so every model describes essentially the same class of functions, giving rise to the opinion that formal computability is both natural and not too narrow.^[2] These functions are sometimes referred to as "recursive", to contrast with the informal term "computable",^[3] a distinction stemming from a 1934 discussion between Kleene and Gödel.^{[4]p.6}

For example, one can formalize computable functions as μ -recursive functions, which are partial functions that take finite tuples of natural numbers and return a single natural number (just as above). They are the smallest class of partial functions that includes the constant, successor, and projection functions, and is closed under composition, primitive recursion, and the μ operator.

Equivalently, computable functions can be formalized as functions which can be calculated by an idealized computing agent such as a Turing machine or a register machine. Formally speaking, a partial function $f : \mathbb{N}^k \rightarrow \mathbb{N}$ can be calculated if and only if there exists a computer program with the following properties:

1. If $f(\mathbf{x})$ is defined, then the program will terminate on the input $\mathbf{x}$ with the value $f(\mathbf{x})$ stored in the computer memory.
2. If $f(\mathbf{x})$ is undefined, then the program never terminates on the input $\mathbf{x}$.

Characteristics of computable functions

The basic characteristic of a computable function is that there must be a finite procedure (an algorithm) telling how to compute the function. The models of computation listed above give different interpretations of what a procedure is and how it is used, but these interpretations share many properties. The fact that these models give equivalent classes of computable functions stems from the fact that each model is capable of reading and mimicking a procedure for any of the other models, much as a compiler is able to read instructions in one computer language and emit instructions in another language.

Enderton [1977] gives the following characteristics of a procedure for computing a computable function; similar characterizations have been given by Turing [1936], Rogers [1967], and others.

- "There must be exact instructions (i.e. a program), finite in length, for the procedure." Thus every computable function must have a finite program that completely describes how the function is to be computed. It is possible to compute the function by just following the instructions; no guessing or special insight is required.
- "If the procedure is given a k -tuple $\mathbf{x}$ in the domain of f , then after a finite number of discrete steps the procedure must terminate and produce $f(\mathbf{x})$." Intuitively, the procedure proceeds step by step, with a specific rule to cover what to do at each step of the calculation. Only finitely many steps can be carried out before the value of the function is returned.
- "If the procedure is given a k -tuple $\mathbf{x}$ which is not in the domain of f , then the procedure might go on forever, never halting. Or it might get stuck at some point (i.e., one of its instructions cannot be executed), but it must not pretend to produce a value for f at $\mathbf{x}$." Thus if a value for $f(\mathbf{x})$ is ever found, it must be the correct value. It is not necessary for the computing agent to distinguish correct outcomes from incorrect ones because the procedure is defined as correct if and only if it produces an outcome.

Enderton goes on to list several clarifications of these 3 requirements of the procedure for a computable function:

1. The procedure must theoretically work for arbitrarily large arguments. It is not assumed that the arguments are smaller than the number of atoms in the Earth, for example.
2. The procedure is required to halt after finitely many steps in order to produce an output, but it may take arbitrarily many steps before halting. No time limitation is assumed.

3. Although the procedure may use only a finite amount of storage space during a successful computation, there is no bound on the amount of space that is used. It is assumed that additional storage space can be given to the procedure whenever the procedure asks for it.

To summarise, based on this view a function is computable if:

- a. given an input from its domain, possibly relying on unbounded storage space, it can give the corresponding output by following a procedure (program, algorithm) that is formed by a finite number of exact unambiguous instructions;
- b. it returns such output (halts) in a finite number of steps; and
- c. if given an input which is not in its domain it either never halts or it gets stuck.

The field of computational complexity studies functions with prescribed bounds on the time and/or space allowed in a successful computation.

Computable sets and relations

A set A of natural numbers is called **computable** (synonyms: **recursive**, **decidable**) if there is a computable, total function f such that for any natural number n , $f(n) = 1$ if n is in A and $f(n) = 0$ if n is not in A .

A set of natural numbers is called **computably enumerable** (synonyms: **recursively enumerable**, **semidecidable**) if there is a computable function f such that for each number n , $f(n)$ is defined if and only if n is in the set. Thus a set is computably enumerable if and only if it is the domain of some computable function. The word *enumerable* is used because the following are equivalent for a nonempty subset B of the natural numbers:

- B is the domain of a computable function.
- B is the range of a total computable function. If B is infinite then the function can be assumed to be injective.

If a set B is the range of a function f then the function can be viewed as an enumeration of B , because the list $f(0)$, $f(1)$, ... will include every element of B .

Because each finitary relation on the natural numbers can be identified with a corresponding set of finite sequences of natural numbers, the notions of **computable relation** and **computably enumerable relation** can be defined from their analogues for sets.

Formal languages

In computability theory in computer science, it is common to consider formal languages. An **alphabet** is an arbitrary set. A **word** on an alphabet is a finite sequence of symbols from the alphabet; the same symbol may be used more than once. For example, binary strings are exactly the words on the alphabet $\{0, 1\}$. A **language** is a subset of the collection of all words on a fixed alphabet. For example, the collection of all binary strings that contain exactly 3 ones is a language over the binary alphabet.

A key property of a formal language is the level of difficulty required to decide whether a given word is in the language. Some coding system must be developed to allow a computable function to take an arbitrary word in the language as input; this is usually considered routine. A language is called **computable** (synonyms: **recursive**, **decidable**) if there is a computable function f such that for each word w over the alphabet, $f(w) = 1$ if the word is in the language and $f(w) = 0$ if the word is not in the language. Thus a language is computable just in case there is a procedure that is able to correctly tell whether arbitrary words are in the language.

A language is **computably enumerable** (synonyms: **recursively enumerable**, **semidecidable**) if there is a computable function f such that $f(w)$ is defined if and only if the word w is in the language. The term *enumerable* has the same etymology as in computably enumerable sets of natural numbers.

Examples

The following functions are computable:

- Each function with a finite domain; e.g., any finite sequence of natural numbers.
- Each constant function $f: \mathbf{N}^k \rightarrow \mathbf{N}, f(n_1, \dots, n_k) := n$.
- Addition $f: \mathbf{N}^2 \rightarrow \mathbf{N}, f(n_1, n_2) := n_1 + n_2$
- The greatest common divisor of two numbers
- A Bézout coefficient of two numbers
- The smallest prime factor of a number

If f and g are computable, then so are: $f + g$, $f * g$, $f \circ g$ if f is unary, $\max(f, g)$, $\min(f, g)$, $\arg \max\{y \leq f(x)\}$ and many more combinations.

The following examples illustrate that a function may be computable though it is not known which algorithm computes it.

- The function f such that $f(n) = 1$ if there is a sequence of *at least* n consecutive fives in the decimal expansion of π , and $f(n) = 0$ otherwise, is computable. (The function f is either the constant 1 function, which is computable, or else there is a k such that $f(n) = 1$ if $n < k$ and $f(n) = 0$ if $n \geq k$. Every such function is computable. It is not known whether there are arbitrarily long runs of fives in the decimal expansion of π , so we don't know *which* of those functions is f . Nevertheless, we know that the function f must be computable.)
- Each finite segment of an *uncomputable* sequence of natural numbers (such as the Busy Beaver function Σ) is computable. E.g., for each natural number n , there exists an algorithm that computes the finite sequence $\Sigma(0), \Sigma(1), \Sigma(2), \dots, \Sigma(n)$ — in contrast to the fact that there is no algorithm that computes the *entire* Σ -sequence, i.e. $\Sigma(n)$ for all n . Thus, "Print 0, 1, 4, 6, 13" is a trivial algorithm to compute $\Sigma(0), \Sigma(1), \Sigma(2), \Sigma(3), \Sigma(4)$; similarly, for any given value of n , such a trivial algorithm *exists* (even though it may never be *known* or produced by anyone) to compute $\Sigma(0), \Sigma(1), \Sigma(2), \dots, \Sigma(n)$.

Church–Turing thesis

The **Church–Turing thesis** states that any function computable from a procedure possessing the three properties listed above is a computable function. Because these three properties are not formally stated, the Church–Turing thesis cannot be proved. The following facts are often taken as evidence for the thesis:

- Many equivalent models of computation are known, and they all give the same definition of computable function (or a weaker version, in some instances).
- No stronger model of computation which is generally considered to be effectively calculable has been proposed.

The Church–Turing thesis is sometimes used in proofs to justify that a particular function is computable by giving a concrete description of a procedure for the computation. This is permitted because it is believed that all such uses of the thesis can be removed by the tedious process of writing a formal procedure for the function in some model of computation.

Provability

Given a function (or, similarly, a set), one may be interested not only if it is computable, but also whether this can be *proven* in a particular proof system (usually first order Peano arithmetic). A function that can be proven to be computable is called **provably total**.

The set of provably total functions is recursively enumerable: one can enumerate all the provably total functions by enumerating all their corresponding proofs, that prove their computability. This can be done by enumerating all the proofs of the proof system and ignoring irrelevant ones.

Relation to recursively defined functions

In a function defined by a recursive definition, each value is defined by a fixed first-order formula of other, previously defined values of the same function or other functions, which might be simply constants. A subset of these is the primitive recursive functions. Another example is the Ackermann function, which is recursively defined but not primitive recursive.^[5]

For definitions of this type to avoid circularity or infinite regress, it is necessary that recursive calls to the same function within a definition be to arguments that are smaller in some well-partial-order on the function's domain. For instance, for the Ackermann function A , whenever the definition of $A(x, y)$ refers to $A(p, q)$, then $(p, q) < (x, y)$ w.r.t. the lexicographic order on pairs of natural numbers. In this case, and in the case of the primitive recursive functions, well-ordering is obvious, but some "refers-to" relations are nontrivial to prove as being well-orderings. Any function defined recursively in a well-ordered way is computable: each value can be computed by expanding a tree of recursive calls to the function, and this expansion must terminate after a finite number of calls, because otherwise Kőnig's lemma would lead to an infinite descending sequence of calls, violating the assumption of well-ordering.

Total functions that are not provably total

In a sound proof system, every provably total function is indeed total, but the converse is not true: in every first-order proof system that is strong enough and sound (including Peano arithmetic), one can prove (in another proof system) the existence of total functions that cannot be proven total in the proof system.

If the total computable functions are enumerated via the Turing machines that produces them, then the above statement can be shown, if the proof system is sound, by a similar diagonalization argument to that used above, using the enumeration of provably total functions given earlier. One uses a Turing machine that enumerates the relevant proofs, and for every input n calls $f_n(n)$ (where f_n is n -th function by *this* enumeration) by invoking the Turing machine that computes it according to the n -th proof. Such a Turing machine is guaranteed to halt if the proof system is sound.

Uncomputable functions and unsolvable problems

Every computable function has a finite procedure giving explicit, unambiguous instructions on how to compute it. Furthermore, this procedure has to be encoded in the finite alphabet used by the computational model, so there are only countably many computable functions. For example, functions may be encoded using a string of bits (the

alphabet $\Sigma = \{0, 1\}$).

The real numbers are uncountable so most real numbers are not computable. See [computable number](#). The set of [finitary](#) functions on the natural numbers is uncountable so most are not computable. Concrete examples of such functions are [Busy beaver](#), [Kolmogorov complexity](#), or any function that outputs the digits of a noncomputable number, such as [Chaitin's constant](#).

Similarly, most subsets of the natural numbers are not computable. The [halting problem](#) was the first such set to be constructed. The [Entscheidungsproblem](#), proposed by [David Hilbert](#), asked whether there is an effective procedure to determine which mathematical statements (coded as natural numbers) are true. Turing and Church independently showed in the 1930s that this set of natural numbers is not computable. According to the Church–Turing thesis, there is no effective procedure (with an algorithm) which can perform these computations.

Extensions of computability

Relative computability

The notion of computability of a function can be [relativized](#) to an arbitrary [set](#) of [natural numbers](#) A . A function f is defined to be **computable in A** (equivalently **A -computable** or **computable relative to A**) when it satisfies the definition of a computable function with modifications allowing access to A as an [oracle](#). As with the concept of a computable function relative computability can be given equivalent definitions in many different models of computation. This is commonly accomplished by supplementing the model of computation with an additional primitive operation which asks whether a given integer is a member of A . We can also talk about f being **computable in g** by identifying g with its graph.

Higher recursion theory

[Hyperarithmetical theory](#) studies those sets that can be computed from a [computable ordinal](#) number of iterates of the [Turing jump](#) of the empty set. This is equivalent to sets defined by both a universal and existential formula in the language of second order arithmetic and to some models of [Hypercomputation](#). Even more general recursion theories have been studied, such as **E-recursion theory** in which any set can be used as an argument to an E-recursive function.

Hyper-computation

Although the Church–Turing thesis states that the computable functions include all functions with algorithms, it is possible to consider broader classes of functions that relax the requirements that algorithms must possess. The field of [Hypercomputation](#) studies models of computation that go beyond normal Turing computation.

See also

- [Computable number](#)
- [Effective method](#)
- [Theory of computation](#)
- [Recursion theory](#)
- [Turing degree](#)
- [Arithmetical hierarchy](#)

- Hypercomputation
- Super-recursive algorithm
- Semicomputable function

References

1. Enderton, Herbert (2002). *A Mathematical Introduction to Logic* (Second ed.). USA: Elsevier. p. 209. ISBN 0-12-238452-0.
 2. Enderton, Herbert (2002). *A Mathematical Introduction to Logic* (Second ed.). USA: Elsevier. p. 208,262. ISBN 0-12-238452-0.
 3. C. J. Ash, J. Knight, *Computable Structures and the Hyperarithmetical Hierarchy* (Studies in Logic and the Foundation of Mathematics, 2000), p. 4
 4. R. Soare, *Computability and Recursion* (<http://www.people.cs.uchicago.edu/~soare/History/compute.pdf>) Archived (<https://web.archive.org/web/20220331221004/http://www.people.cs.uchicago.edu/~soare/History/compute.pdf>) 2022-03-31 at the Wayback Machine (1995). Accessed 9 November 2022.
 5. Péter, Rózsa (1935). "Konstruktion nichtrekursiver Funktionen". *Mathematische Annalen*. **111**: 42–60. doi:10.1007/BF01472200 (<https://doi.org/10.1007%2FBF01472200>). S2CID 121107217 (<http://api.semanticscholar.org/CorpusID:121107217>).
- Cutland, Nigel. *Computability*. Cambridge University Press, 1980.
 - Enderton, H.B. Elements of recursion theory. *Handbook of Mathematical Logic* (North-Holland 1977) pp. 527–566.
 - Rogers, H. *Theory of recursive functions and effective computation* (McGraw–Hill 1967).
 - Turing, A. (1937), *On Computable Numbers, With an Application to the Entscheidungsproblem* (<https://academic.oup.com/plms/issue/s2-42/1>). *Proceedings of the London Mathematical Society*, Series 2, Volume 42 (1937), p.230–265. Reprinted in M. Davis (ed.), *The Undecidable*, Raven Press, Hewlett, NY, 1965.

This article is issued from Wikipedia. The text is available under Creative Commons Attribution-Share Alike 4.0 unless otherwise noted. Additional terms may apply for the media files.